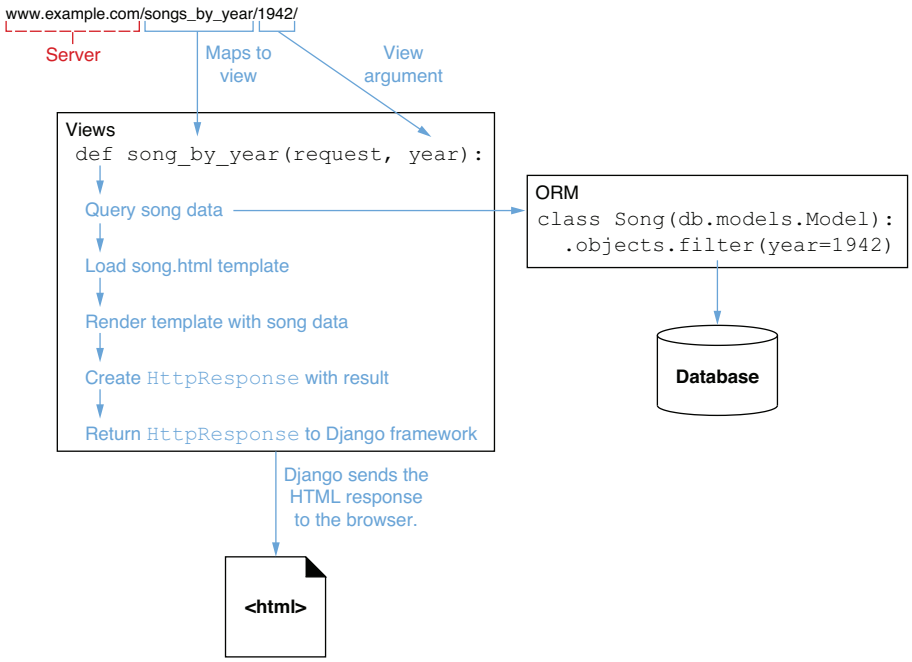


Figure 1.3 A song-by-year view rendering a web page based on an argument

and-feel for the website. This means songs.html only contains the parts that are unique to that page. The template engine has a tag language, which includes a feature that lets you loop over content, generating the rows in an HTML `<table>`, populating each row with the information from a song from 1942. You'll learn all about Django templates, their tags, and how to compose them together in chapter 3.

Once the HTML has been rendered, it gets packaged in an `HttpResponse` object and sent back to the Django framework. The loading and rendering of a template is so common that Django provides shortcuts to do this.

Django uses the response object to construct the data the web server needs for the browser. This includes all the HTTP headers and the body containing the HTML. The content gets returned to the web server through WSGI, and the server sends it down to the browser. That's how you see your page full of songs.

1.1.1 Mapping URLs, Django views, and the MVC model

The simplest form of any website is a mapping between a file and a URL. A web page that shows a picture of a kitten is at least two files: one for the HTML document and one for the image of the kitten. Simple file-to-URL mapped sites like this are called *static sites*. If you want your users to be able to upload their own images or add comments to your page, your site can no longer be just static files. A *dynamic site* is one that determines a page's content when it gets visited.